SWING WITH DATABASE CONNECTIVITY

LIKHITA ACHARYA

2401001

Q1) Assignment 1: Create a Login System

- **Description: Build a login screen using Java Swing, where the user can enter their username and password. Upon clicking the "Login" button:**
    - **The application should validate the credentials by checking the user information stored in a database (e.g., MySQL or SQLite).**
    - **If the credentials are correct, show a success message and navigate to a new screen (or show a new panel).**
    - **If the credentials are incorrect, display an error message.**

LoginPage.java

```
package assign1pkg;


import java.awt.EventQueue;


import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JLabel;
import javax.swing.JOptionPane;


import java.awt.Font;
import javax.swing.JTextField;
import javax.swing.JPasswordField;
import javax.swing.JButton;
import java.awt.event.ActionListener;
```

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.awt.event.ActionEvent;

public class LoginPage extends JFrame {

	private static final long serialVersionUID = 1L;
	private JPanel contentPane;
	private JTextField textField;
	private JPasswordField passwordField;

	/**
	 * Launch the application.
	 */
	public static void main(String[] args) {
		EventQueue.invokeLater(new Runnable() {
			public void run() {
				try {
					LoginPage frame = new LoginPage();
					frame.setVisible(true);
				} catch (Exception e) {
					e.printStackTrace();
				}
			}
		});
	}
```

```
/**
 * Create the frame.
 */
public LoginPage() {
	setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
	setBounds(100, 100, 776, 633);
	contentPane = new JPanel();
	contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));

	setContentPane(contentPane);
	contentPane.setLayout(null);

	JLabel lblNewLabel = new JLabel("Username");
	lblNewLabel.setFont(new Font("Tahoma", Font.BOLD, 18));
	lblNewLabel.setBounds(58, 47, 116, 39);
	contentPane.add(lblNewLabel);

	JLabel lblPassword = new JLabel("Password");
	lblPassword.setFont(new Font("Tahoma", Font.BOLD, 18));
	lblPassword.setBounds(58, 101, 116, 39);
	contentPane.add(lblPassword);

	textField = new JTextField();
	textField.setBounds(201, 49, 195, 30);
	contentPane.add(textField);
	textField.setColumns(10);
```

```
passwordField = new JPasswordField();
passwordField.setBounds(201, 103, 195, 30);
contentPane.add(passwordField);

JButton btnNewButton = new JButton("Login");
btnNewButton.addActionListener(new ActionListener() {
	public void actionPerformed(ActionEvent e) {
		String username = textField.getText();
		String password = new String(passwordField.getPassword());

		if (username.isEmpty() || password.isEmpty()) {
			JOptionPane.showMessageDialog(null, "Please enter
both username and password.");
			return;
		}

		try {
			Connection conn = DBConnection.getConnection();
			String sql = "SELECT * FROM users WHERE username =
? AND password = ?";
			PreparedStatement stmt =
conn.prepareStatement(sql);
			stmt.setString(1, username);
			stmt.setString(2, password);

			ResultSet rows = stmt.executeQuery();
			if (rows.next()) {
				JOptionPane.showMessageDialog(null, "Login
successful!");
```

```
                    } else {
                        JOptionPane.showMessageDialog(null, "Invalid
username or password.");
                    }
                    stmt.close();
                    conn.close();
                    rows.close();
                } catch (SQLException ex) {
                }
            }

        });
        btnNewButton.setFont(new Font("Tahoma", Font.BOLD, 16));
        btnNewButton.setBounds(171, 181, 108, 30);
        contentPane.add(btnNewButton);
    }
}
```

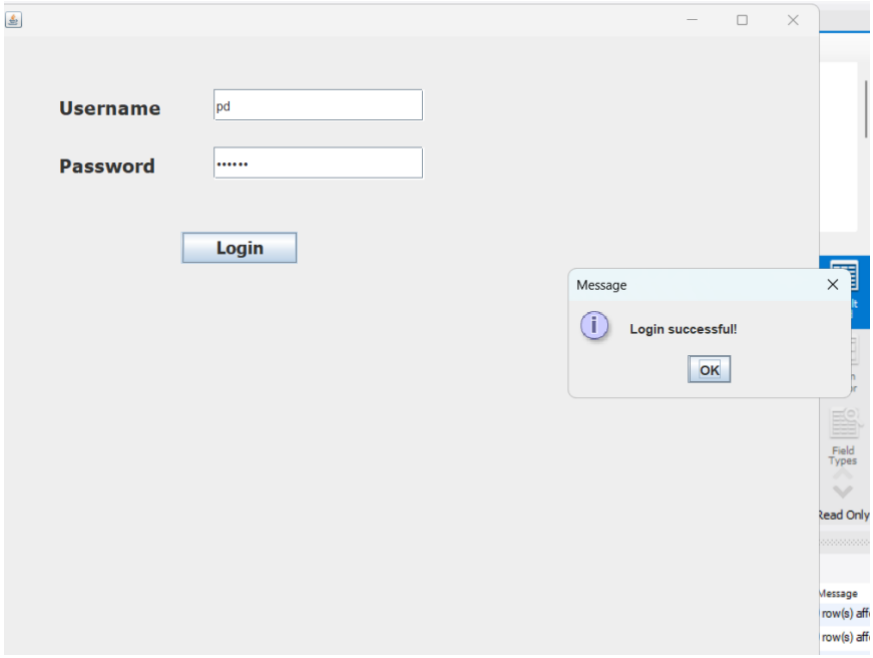
DBConnection.java

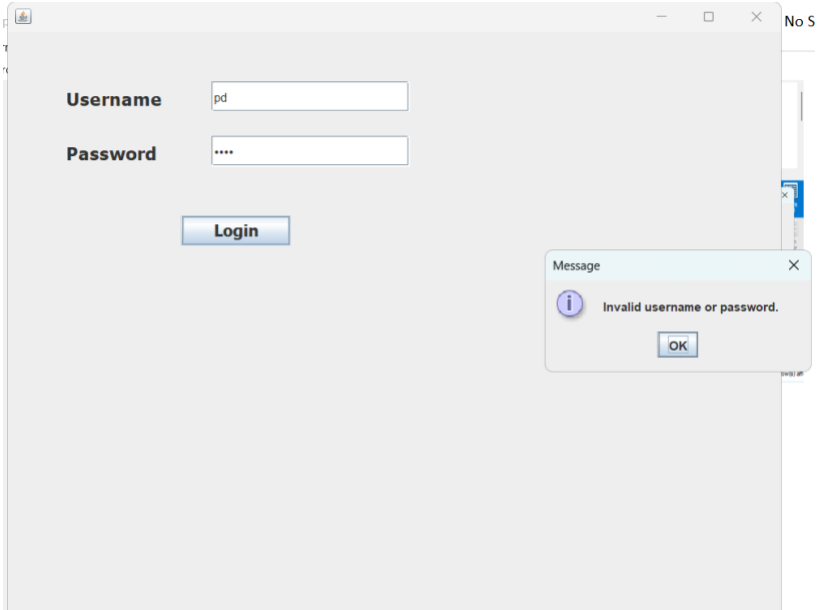
```
package assign1pkg;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class DBConnection {
    public static void main(String[] args) {
        String jdbcUrl="jdbc:mysql://localhost:3306/user1";
```

```
		String username="root";
		String password="admin@123";

		try {
			Connection connection= DriverManager.getConnection(jdbcUrl,
username, password);
			Statement statement=connection.createStatement();
			System.out.println("successfully connected");

		}
		catch(SQLException e) {
			System.err.println("error:"+e.getMessage());
			e.printStackTrace();
		}

	}

	public static Connection getConnection() {
		// TODO Auto-generated method stub
//		return null;
		String jdbcUrl = "jdbc:mysql://localhost:3306/user1";
	    String username = "root";
	    String password = "admin@123";

	    try {
	        return DriverManager.getConnection(jdbcUrl, username, password);
	    } catch (SQLException e) {
	        System.err.println("Connection Error: " + e.getMessage());
```

```
            e.printStackTrace();
            return null;
        }
    }

}
```

Output:





Q2) Assignment 2: CRUD Operations for a Student Management System

- **Description: Develop a simple student management system with the following features:**
    - **Add new student details (name, roll number, course).**
    - **View the list of students in a JTable.**
    - **Update student details.**
    - **Delete student records.**

StudentMgtSystem.java

```
package assign2pkg;


import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.awt.event.*;

import java.sql.*;


public class StudentMgtSystem extends JFrame {

   private JTextField nameField, rollField, courseField;
```

```
private JTable table;
private DefaultTableModel model;

Connection conn;

public StudentMgtSystem() {
    setTitle("Student Management System");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 800, 600);
    getContentPane().setLayout(null);

    JLabel lblTitle = new JLabel("Student Management System");
    lblTitle.setFont(new Font("Tahoma", Font.BOLD, 24));
    lblTitle.setBounds(220, 10, 400, 40);
    getContentPane().add(lblTitle);

    JLabel lblName = new JLabel("Name:");
    lblName.setBounds(30, 80, 100, 25);
    getContentPane().add(lblName);

    nameField = new JTextField();
    nameField.setBounds(130, 80, 200, 25);
    getContentPane().add(nameField);

    JLabel lblRoll = new JLabel("Roll No:");
    lblRoll.setBounds(30, 120, 100, 25);
    getContentPane().add(lblRoll);
```

```
rollField = new JTextField();
rollField.setBounds(130, 120, 200, 25);
getContentPane().add(rollField);

JLabel lblCourse = new JLabel("Course:");
lblCourse.setBounds(30, 160, 100, 25);
getContentPane().add(lblCourse);

courseField = new JTextField();
courseField.setBounds(130, 160, 200, 25);
getContentPane().add(courseField);

JButton btnAdd = new JButton("Add");
btnAdd.setBounds(30, 210, 80, 30);
getContentPane().add(btnAdd);

JButton btnUpdate = new JButton("Update");
btnUpdate.setBounds(120, 210, 100, 30);
getContentPane().add(btnUpdate);

JButton btnDelete = new JButton("Delete");
btnDelete.setBounds(230, 210, 100, 30);
getContentPane().add(btnDelete);

JScrollPane scrollPane = new JScrollPane();
scrollPane.setBounds(350, 80, 400, 300);
getContentPane().add(scrollPane);
```

```
table = new JTable();
model = new DefaultTableModel(new String[]{"ID", "Name", "Roll No", "Course"}, 0);
table.setModel(model);
scrollPane.setViewportView(table);

connectDatabase();
loadTableData();

// Add Student
btnAdd.addActionListener(e -> {
    try {
        String sql = "INSERT INTO students(name, roll_number, course) VALUES (?, ?, ?)";
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setString(1, nameField.getText());
        ps.setString(2, rollField.getText());
        ps.setString(3, courseField.getText());
        ps.executeUpdate();
        JOptionPane.showMessageDialog(this, "Student Added Successfully!");
        clearFields();
        loadTableData();
    } catch (Exception ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
    }
});

// Update Student
btnUpdate.addActionListener(e -> {
```

```
        int row = table.getSelectedRow();
        if (row >= 0) {
            int id = Integer.parseInt(model.getValueAt(row, 0).toString());
            try {
                String sql = "UPDATE students SET name=?, roll_number=?, course=? WHERE
id=?";
                PreparedStatement ps = conn.prepareStatement(sql);
                ps.setString(1, nameField.getText());
                ps.setString(2, rollField.getText());
                ps.setString(3, courseField.getText());
                ps.setInt(4, id);
                ps.executeUpdate();
                JOptionPane.showMessageDialog(this, "Student Updated Successfully!");
                clearFields();
                loadTableData();
            } catch (Exception ex) {
                ex.printStackTrace();
                JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
            }
        } else {
            JOptionPane.showMessageDialog(this, "Please select a student to update.");
        }
    });

    // Delete Student
    btnDelete.addActionListener(e -> {
        int row = table.getSelectedRow();
        if (row >= 0) {
```

```
            int id = Integer.parseInt(model.getValueAt(row, 0).toString());
            try {
                String sql = "DELETE FROM students WHERE id=?";
                PreparedStatement ps = conn.prepareStatement(sql);
                ps.setInt(1, id);
                ps.executeUpdate();
                JOptionPane.showMessageDialog(this, "Student Deleted Successfully!");
                clearFields();
                loadTableData();
            } catch (Exception ex) {
                ex.printStackTrace();
                JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
            }
        } else {
            JOptionPane.showMessageDialog(this, "Please select a student to delete.");
        }
    });

    // Table row selection
    table.addMouseListener(new MouseAdapter() {
        public void mouseClicked(MouseEvent e) {
            int row = table.getSelectedRow();
            nameField.setText(model.getValueAt(row, 1).toString());
            rollField.setText(model.getValueAt(row, 2).toString());
            courseField.setText(model.getValueAt(row, 3).toString());
        }
    });
}
```

```
    private void connectDatabase() {
        try {
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/user1", "root",
"admin@123");
        } catch (Exception e) {
            e.printStackTrace();
            JOptionPane.showMessageDialog(this, "Database connection failed!");
        }
    }

    private void loadTableData() {
        model.setRowCount(0);
        try {
            String sql = "SELECT * FROM students";
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getString("roll_number"),
                    rs.getString("course")
                });
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
```

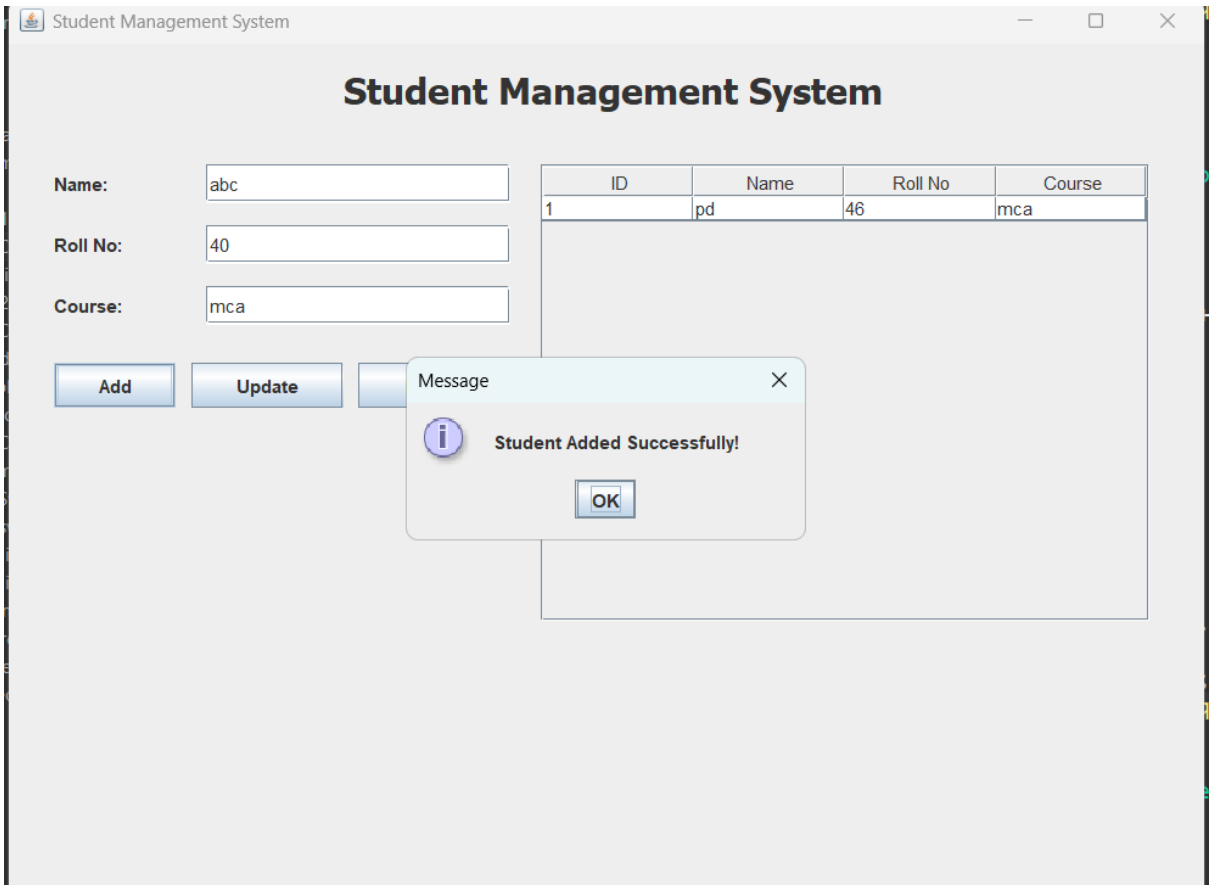
```
    }

    private void clearFields() {
        nameField.setText("");
        rollField.setText("");
        courseField.setText("");
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> {
            try {
                StudentMgtSystem frame = new StudentMgtSystem();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }
}
```

DBConnection.java

```
package assign2pkg;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    public static Connection getConnection() {
```

```
        try {
            return DriverManager.getConnection("jdbc:mysql://localhost:3306/user1", "root",
"admin@123");
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Output:



Q3) Assignment 3: Library Book Inventory System

- **Description: Create a library book inventory system that allows the following functionalities:**

- **Add a new book with details like title, author, ISBN, and quantity.**
- **View available books in the library using a JTable.**
- **Update the book information (e.g., change the quantity of books).**
- **Delete a book from the inventory.**
- **Search for books by title or author.**

BookInventory.java

```
package assign3pkg;


import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;


public class BookInventory extends JFrame {
    private JTextField titleField, authorField, isbnField, quantityField, searchField;
    private JTable table;
    private DefaultTableModel model;

    public BookInventory() {
        setTitle("Library Book Inventory");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(900, 600);
        setLayout(null);

        JLabel lblTitle = new JLabel("Title:");
        lblTitle.setBounds(20, 20, 80, 25);
        add(lblTitle);
```

```
titleField = new JTextField();
titleField.setBounds(100, 20, 150, 25);
add(titleField);

JLabel lblAuthor = new JLabel("Author:");
lblAuthor.setBounds(280, 20, 80, 25);
add(lblAuthor);
authorField = new JTextField();
authorField.setBounds(360, 20, 150, 25);
add(authorField);

JLabel lblISBN = new JLabel("ISBN:");
lblISBN.setBounds(530, 20, 80, 25);
add(lblISBN);
isbnField = new JTextField();
isbnField.setBounds(600, 20, 120, 25);
add(isbnField);

JLabel lblQuantity = new JLabel("Qty:");
lblQuantity.setBounds(730, 20, 50, 25);
add(lblQuantity);
quantityField = new JTextField();
quantityField.setBounds(780, 20, 50, 25);
add(quantityField);

JButton addBtn = new JButton("Add");
addBtn.setBounds(100, 60, 80, 25);
add(addBtn);
```

```
JButton updateBtn = new JButton("Update");
updateBtn.setBounds(200, 60, 100, 25);
add(updateBtn);

JButton deleteBtn = new JButton("Delete");
deleteBtn.setBounds(320, 60, 100, 25);
add(deleteBtn);

JButton clearBtn = new JButton("Clear");
clearBtn.setBounds(440, 60, 100, 25);
add(clearBtn);

JLabel lblSearch = new JLabel("Search:");
lblSearch.setBounds(570, 60, 60, 25);
add(lblSearch);
searchField = new JTextField();
searchField.setBounds(630, 60, 150, 25);
add(searchField);

JButton searchBtn = new JButton("Search");
searchBtn.setBounds(790, 60, 80, 25);
add(searchBtn);

model = new DefaultTableModel(new String[]{"ID", "Title", "Author", "ISBN",
"Quantity"}, 0);
table = new JTable(model);
JScrollPane scrollPane = new JScrollPane(table);
```

```
        scrollPane.setBounds(20, 100, 850, 400);
        add(scrollPane);

        // Event Listeners
        addBtn.addActionListener(e -> addBook());
        updateBtn.addActionListener(e -> updateBook());
        deleteBtn.addActionListener(e -> deleteBook());
        clearBtn.addActionListener(e -> clearForm());
        searchBtn.addActionListener(e -> searchBooks());

        table.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                int row = table.getSelectedRow();
                titleField.setText(model.getValueAt(row, 1).toString());
                authorField.setText(model.getValueAt(row, 2).toString());
                isbnField.setText(model.getValueAt(row, 3).toString());
                quantityField.setText(model.getValueAt(row, 4).toString());
            }
        });

        loadBooks();
        setVisible(true);
    }

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/user1", "root",
"admin@123");
    }
```

```
private void addBook() {
    try (Connection conn = getConnection()) {
        String query = "INSERT INTO books (title, author, isbn, quantity) VALUES (?, ?, ?, ?)";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setString(1, titleField.getText());
        ps.setString(2, authorField.getText());
        ps.setString(3, isbnField.getText());
        ps.setInt(4, Integer.parseInt(quantityField.getText()));
        ps.executeUpdate();
        JOptionPane.showMessageDialog(this, "Book added successfully!");
        loadBooks();
        clearForm();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private void updateBook() {
    int row = table.getSelectedRow();
    if (row < 0) return;
    int id = Integer.parseInt(model.getValueAt(row, 0).toString());

    try (Connection conn = getConnection()) {
        String query = "UPDATE books SET title=?, author=?, isbn=?, quantity=? WHERE id=?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setString(1, titleField.getText());
        ps.setString(2, authorField.getText());
```

```
        ps.setString(3, isbnField.getText());
        ps.setInt(4, Integer.parseInt(quantityField.getText()));
        ps.setInt(5, id);
        ps.executeUpdate();
        JOptionPane.showMessageDialog(this, "Book updated successfully!");
        loadBooks();
        clearForm();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private void deleteBook() {
    int row = table.getSelectedRow();
    if (row < 0) return;
    int id = Integer.parseInt(model.getValueAt(row, 0).toString());

    try (Connection conn = getConnection()) {
        String query = "DELETE FROM books WHERE id=?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, id);
        ps.executeUpdate();
        JOptionPane.showMessageDialog(this, "Book deleted!");
        loadBooks();
        clearForm();
    } catch (Exception e) {
        e.printStackTrace();
    }
```

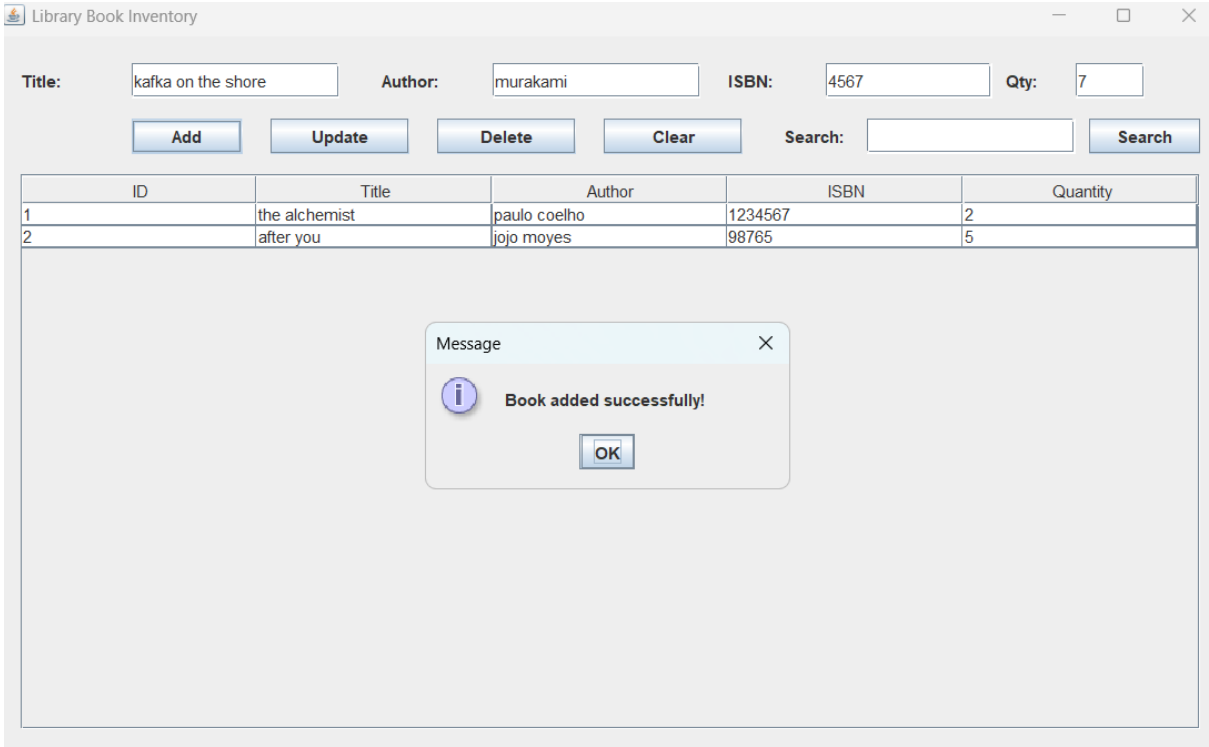
```
}

private void searchBooks() {
    String keyword = searchField.getText();
    model.setRowCount(0);
    try (Connection conn = getConnection()) {
        String query = "SELECT * FROM books WHERE title LIKE ? OR author LIKE ?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setString(1, "%" + keyword + "%");
        ps.setString(2, "%" + keyword + "%");
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("title"),
                rs.getString("author"),
                rs.getString("isbn"),
                rs.getInt("quantity")
            });
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private void loadBooks() {
    model.setRowCount(0);
    try (Connection conn = getConnection()) {
```

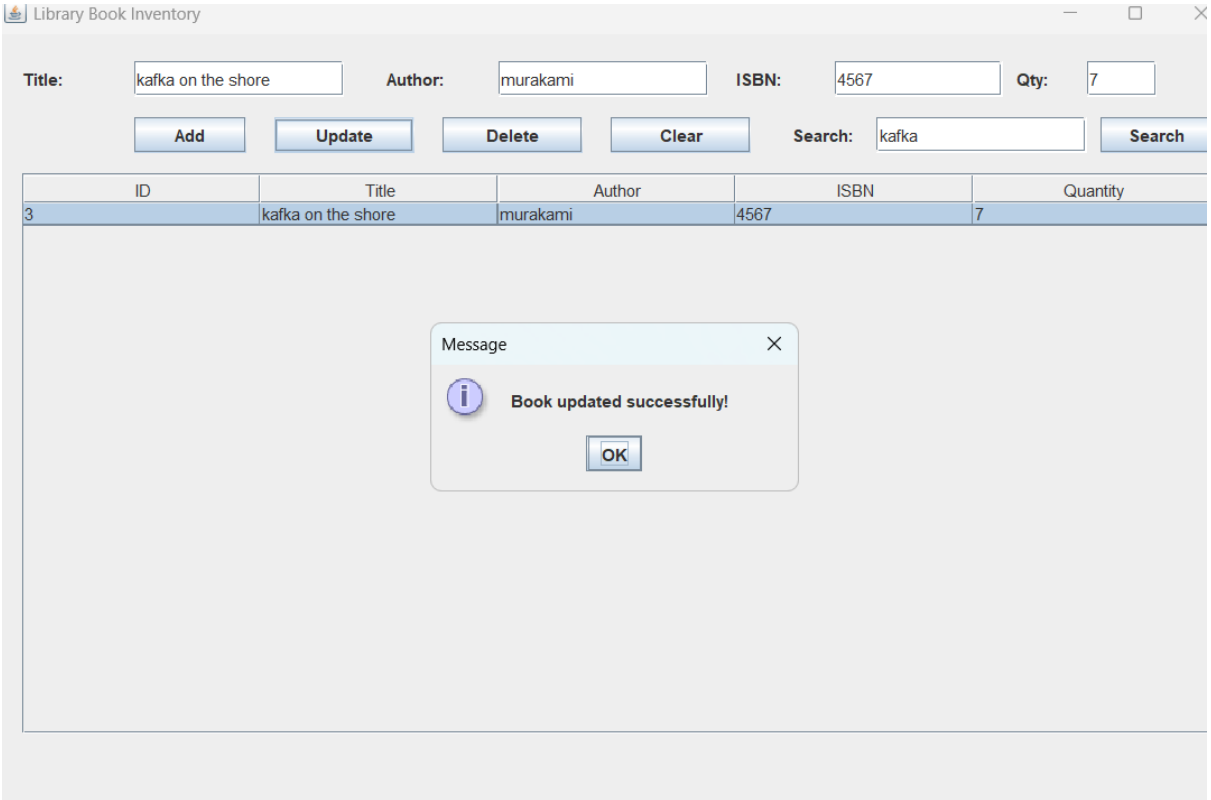
```
        String query = "SELECT * FROM books";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("title"),
                rs.getString("author"),
                rs.getString("isbn"),
                rs.getInt("quantity")
            });
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private void clearForm() {
    titleField.setText("");
    authorField.setText("");
    isbnField.setText("");
    quantityField.setText("");
    table.clearSelection();
}

public static void main(String[] args) {
    new BookInventory();
}
```

```
}
```

DBConnection.java

```
package assign3pkg;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/user1";
    private static final String USER = "root
    private static final String PASSWORD = "admin@123";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```





Assignment 4: Employee Management System

- **Description: Build an employee management system with the following features:**

- **Add a new employee to the database (employee ID, name, department, salary).**
- **Display all employees in a JTable.**
- **Update employee details (e.g., salary increase, department change).**
- **Delete an employee record.**
- **Calculate the total salary of all employees and display it in a label.**

EmployeeManagementSystem.java

```
package assign4pkg;


import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;


public class EmployeeManagementSystem extends JFrame {
    private JTextField txtId, txtName, txtDept, txtSalary;
    private JTable table;
    private DefaultTableModel model;
    private JLabel lblTotalSalary;


    public EmployeeManagementSystem() {
        setTitle("Employee Management System");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);
```

```
JLabel lbl1 = new JLabel("Employee ID:");
lbl1.setBounds(20, 20, 100, 25);
add(lbl1);

txtId = new JTextField();
txtId.setBounds(130, 20, 150, 25);
add(txtId);

JLabel lbl2 = new JLabel("Name:");
lbl2.setBounds(20, 60, 100, 25);
add(lbl2);

txtName = new JTextField();
txtName.setBounds(130, 60, 150, 25);
add(txtName);

JLabel lbl3 = new JLabel("Department:");
lbl3.setBounds(20, 100, 100, 25);
add(lbl3);

txtDept = new JTextField();
txtDept.setBounds(130, 100, 150, 25);
add(txtDept);

JLabel lbl4 = new JLabel("Salary:");
lbl4.setBounds(20, 140, 100, 25);
add(lbl4);
```

```
txtSalary = new JTextField();
txtSalary.setBounds(130, 140, 150, 25);
add(txtSalary);

JButton btnAdd = new JButton("Add");
btnAdd.setBounds(20, 180, 80, 30);
add(btnAdd);

JButton btnUpdate = new JButton("Update");
btnUpdate.setBounds(110, 180, 90, 30);
add(btnUpdate);

JButton btnDelete = new JButton("Delete");
btnDelete.setBounds(210, 180, 90, 30);
add(btnDelete);

JButton btnSalary = new JButton("Total Salary");
btnSalary.setBounds(310, 180, 120, 30);
add(btnSalary);

lblTotalSalary = new JLabel("Total: 0.0");
lblTotalSalary.setBounds(450, 180, 200, 30);
add(lblTotalSalary);

model = new DefaultTableModel();
table = new JTable(model);
model.addColumn("ID");
model.addColumn("Name");
```

```
    model.addColumn("Department");
    model.addColumn("Salary");

    JScrollPane scroll = new JScrollPane(table);
    scroll.setBounds(20, 230, 740, 300);
    add(scroll);

    loadTable();

    btnAdd.addActionListener(e -> addEmployee());
    btnUpdate.addActionListener(e -> updateEmployee());
    btnDelete.addActionListener(e -> deleteEmployee());
    btnSalary.addActionListener(e -> calculateTotalSalary());

    table.addMouseListener(new MouseAdapter() {
        public void mouseClicked(MouseEvent e) {
            int row = table.getSelectedRow();
            txtId.setText(model.getValueAt(row, 0).toString());
            txtName.setText(model.getValueAt(row, 1).toString());
            txtDept.setText(model.getValueAt(row, 2).toString());
            txtSalary.setText(model.getValueAt(row, 3).toString());
        }
    });
}

private void addEmployee() {
    try (Connection conn = DBConnection.getConnection()) {
        String query = "INSERT INTO employees VALUES (?, ?, ?, ?)";
```

```
            PreparedStatement ps = conn.prepareStatement(query);
            ps.setInt(1, Integer.parseInt(txtId.getText()));
            ps.setString(2, txtName.getText());
            ps.setString(3, txtDept.getText());
            ps.setDouble(4, Double.parseDouble(txtSalary.getText()));
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Employee Added");
            clearFields();
            loadTable();
        } catch (SQLException ex) {
            JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
        }
    }

    private void updateEmployee() {
        try (Connection conn = DBConnection.getConnection()) {
            String query = "UPDATE employees SET name=?, department=?, salary=? WHERE
id=?";
            PreparedStatement ps = conn.prepareStatement(query);
            ps.setString(1, txtName.getText());
            ps.setString(2, txtDept.getText());
            ps.setDouble(3, Double.parseDouble(txtSalary.getText()));
            ps.setInt(4, Integer.parseInt(txtId.getText()));
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Employee Updated");
            clearFields();
            loadTable();
        } catch (SQLException ex) {
```

```
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
    }
}

private void deleteEmployee() {
    try (Connection conn = DBConnection.getConnection()) {
        String query = "DELETE FROM employees WHERE id=?";
        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, Integer.parseInt(txtId.getText()));
        ps.executeUpdate();
        JOptionPane.showMessageDialog(this, "Employee Deleted");
        clearFields();
        loadTable();
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
    }
}

private void calculateTotalSalary() {
    try (Connection conn = DBConnection.getConnection()) {
        String query = "SELECT SUM(salary) FROM employees";
        PreparedStatement ps = conn.prepareStatement(query);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            lblTotalSalary.setText("Total: " + rs.getDouble(1));
        }
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
```

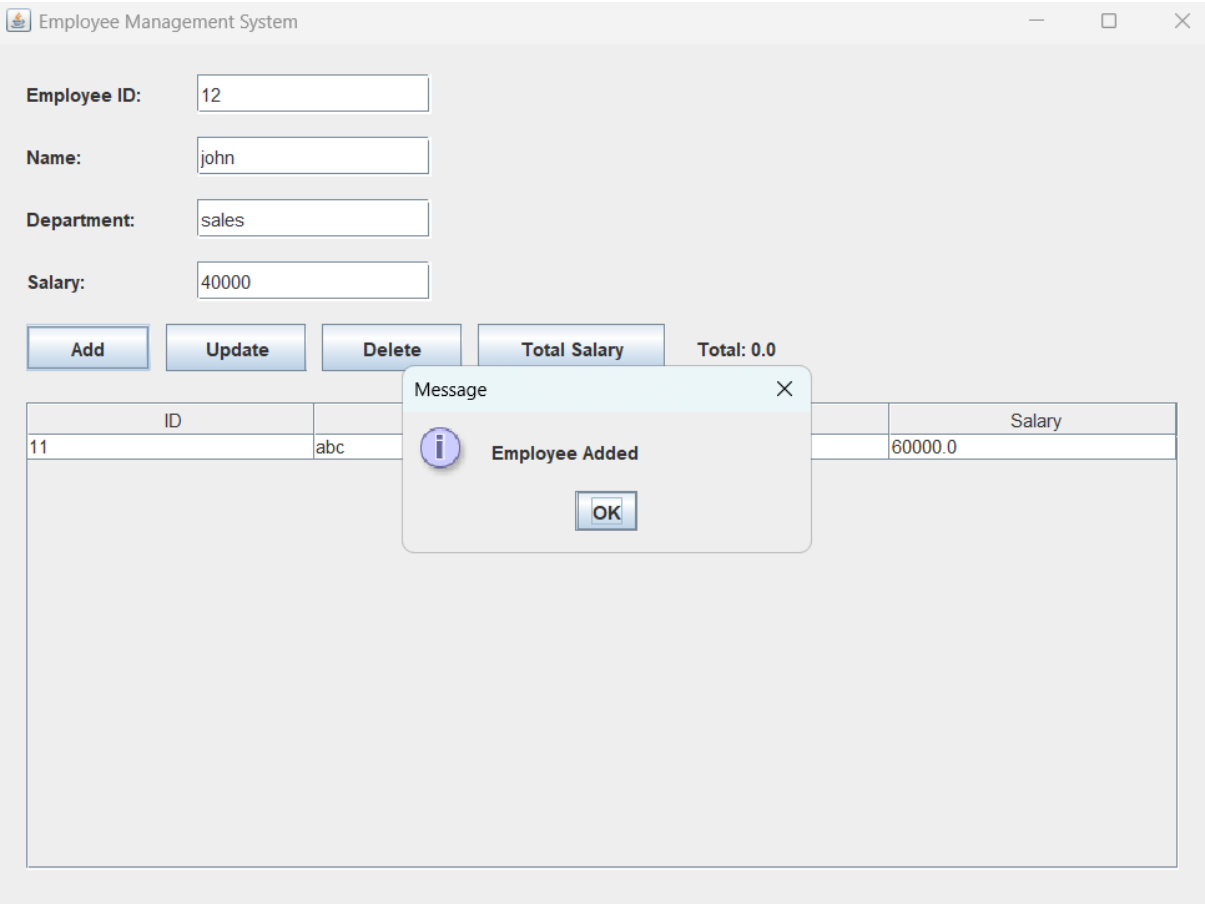
```
    }
}

private void loadTable() {
    model.setRowCount(0);
    try (Connection conn = DBConnection.getConnection()) {
        String query = "SELECT * FROM employees";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("department"),
                rs.getDouble("salary")
            });
        }
    } catch (SQLException ex) {
        JOptionPane.showMessageDialog(this, "Load Error: " + ex.getMessage());
    }
}

private void clearFields() {
    txtId.setText("");
    txtName.setText("");
    txtDept.setText("");
    txtSalary.setText("");
}
```

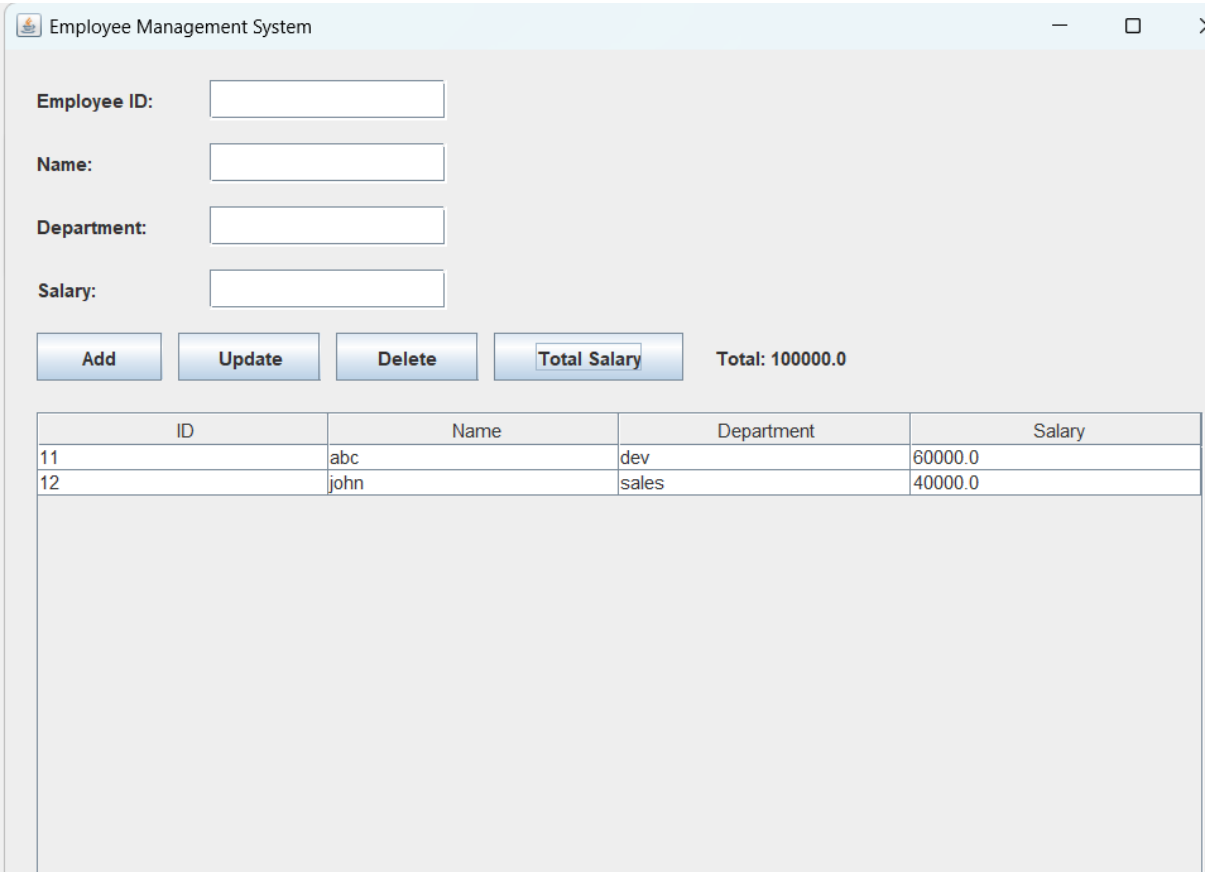
```
    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new EmployeeManagementSystem().setVisible(true));
    }
}
```

DBConnection.java

```
package assign4pkg;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    public static Connection getConnection() {
        try {
            String url = "jdbc:mysql://localhost:3306/employee_db";
            String user = "root";
            String pass = "admin@123"; // Replace with your password
            return DriverManager.getConnection(url, user, pass);
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Output:


Employee Management System
Employee ID:
12
Name:
john
Department:
sales
Salary:
40000
Add
Update
Delete
Total Salary
Total: 0.0
Message
Employee Added
OK
ID
Salary
11
abc
60000.0


Employee Management System
Employee ID:
Name:
Department:
Salary:
Add
Update
Delete
Total Salary
Total: 100000.0
ID
Name
Department
Salary
11
abc
dev
60000.0
12
john
sales
40000.0

Assignment 5 : Invoice Generation System

- **Description: Create an invoice generation system for a shop where customers can place orders. The system should:**
    - **Allow the user to add multiple items (product name, price, quantity).**
    - **Calculate the total cost of the items (price * quantity).**
    - **Generate and save the invoice to the database.**
    - **Display a list of previous invoices in a JTable with options to view individual invoices.**

InvoiceSystem.java

```
package assign5pkg;


import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;


public class InvoiceSystem extends JFrame {
    private JTextField txtProduct, txtPrice, txtQuantity;
    private JLabel lblTotal;
    private JTable table;
    private DefaultTableModel model;
    private double total = 0.0;

    Connection conn;

    public InvoiceSystem() {
        setTitle("Invoice Generation System");
        setSize(800, 600);
```

```
setLocationRelativeTo(null);
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLayout(null);

JLabel lbl1 = new JLabel("Product:");
lbl1.setBounds(30, 30, 80, 25);
add(lbl1);

txtProduct = new JTextField();
txtProduct.setBounds(120, 30, 150, 25);
add(txtProduct);

JLabel lbl2 = new JLabel("Price:");
lbl2.setBounds(30, 70, 80, 25);
add(lbl2);

txtPrice = new JTextField();
txtPrice.setBounds(120, 70, 150, 25);
add(txtPrice);

JLabel lbl3 = new JLabel("Quantity:");
lbl3.setBounds(30, 110, 80, 25);
add(lbl3);

txtQuantity = new JTextField();
txtQuantity.setBounds(120, 110, 150, 25);
add(txtQuantity);
```

```
        JButton btnAdd = new JButton("Add Item");
        btnAdd.setBounds(30, 150, 100, 30);
        add(btnAdd);

        lblTotal = new JLabel("Total: ₹0.0");
        lblTotal.setFont(new Font("Arial", Font.BOLD, 16));
        lblTotal.setBounds(30, 200, 300, 30);
        add(lblTotal);

        model = new DefaultTableModel(new String[]{"Product", "Price", "Quantity", "Total"},
0);
        table = new JTable(model);
        JScrollPane scrollPane = new JScrollPane(table);
        scrollPane.setBounds(300, 30, 460, 300);
        add(scrollPane);

        JButton btnSave = new JButton("Save Invoice");
        btnSave.setBounds(30, 250, 150, 30);
        add(btnSave);

        JButton btnView = new JButton("View Invoices");
        btnView.setBounds(200, 250, 150, 30);
        add(btnView);

        connect();

        btnAdd.addActionListener(e -> {
            String product = txtProduct.getText();
```

```
            double price = Double.parseDouble(txtPrice.getText());
            int qty = Integer.parseInt(txtQuantity.getText());
            double lineTotal = price * qty;
            total += lineTotal;

            model.addRow(new Object[]{product, price, qty, lineTotal});
            lblTotal.setText("Total: ₹" + total);

            txtProduct.setText("");
            txtPrice.setText("");
            txtQuantity.setText("");
        });

        btnSave.addActionListener(e -> saveInvoice());

        btnView.addActionListener(e -> viewInvoices());
    }

    public void connect() {
        try {
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/user1", "root",
"admin@123");
            System.out.println("DB Connected");
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Database Connection Failed: " +
e.getMessage());
        }
    }
```

```
    public void saveInvoice() {
        try {
            String sql = "INSERT INTO invoices (product_name, price, quantity, total) VALUES (?, ?,
?, ?)";
            PreparedStatement ps = conn.prepareStatement(sql);

            for (int i = 0; i < table.getRowCount(); i++) {
                ps.setString(1, table.getValueAt(i, 0).toString());
                ps.setDouble(2, Double.parseDouble(table.getValueAt(i, 1).toString()));
                ps.setInt(3, Integer.parseInt(table.getValueAt(i, 2).toString()));
                ps.setDouble(4, Double.parseDouble(table.getValueAt(i, 3).toString()));
                ps.executeUpdate();
            }

            JOptionPane.showMessageDialog(this, "Invoice Saved Successfully");
            model.setRowCount(0);
            total = 0.0;
            lblTotal.setText("Total: ₹0.0");

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error saving invoice: " + e.getMessage());
        }
    }

    public void viewInvoices() {
        JFrame viewFrame = new JFrame("Invoice Records");
        viewFrame.setSize(600, 400);
        viewFrame.setLocationRelativeTo(null);
```

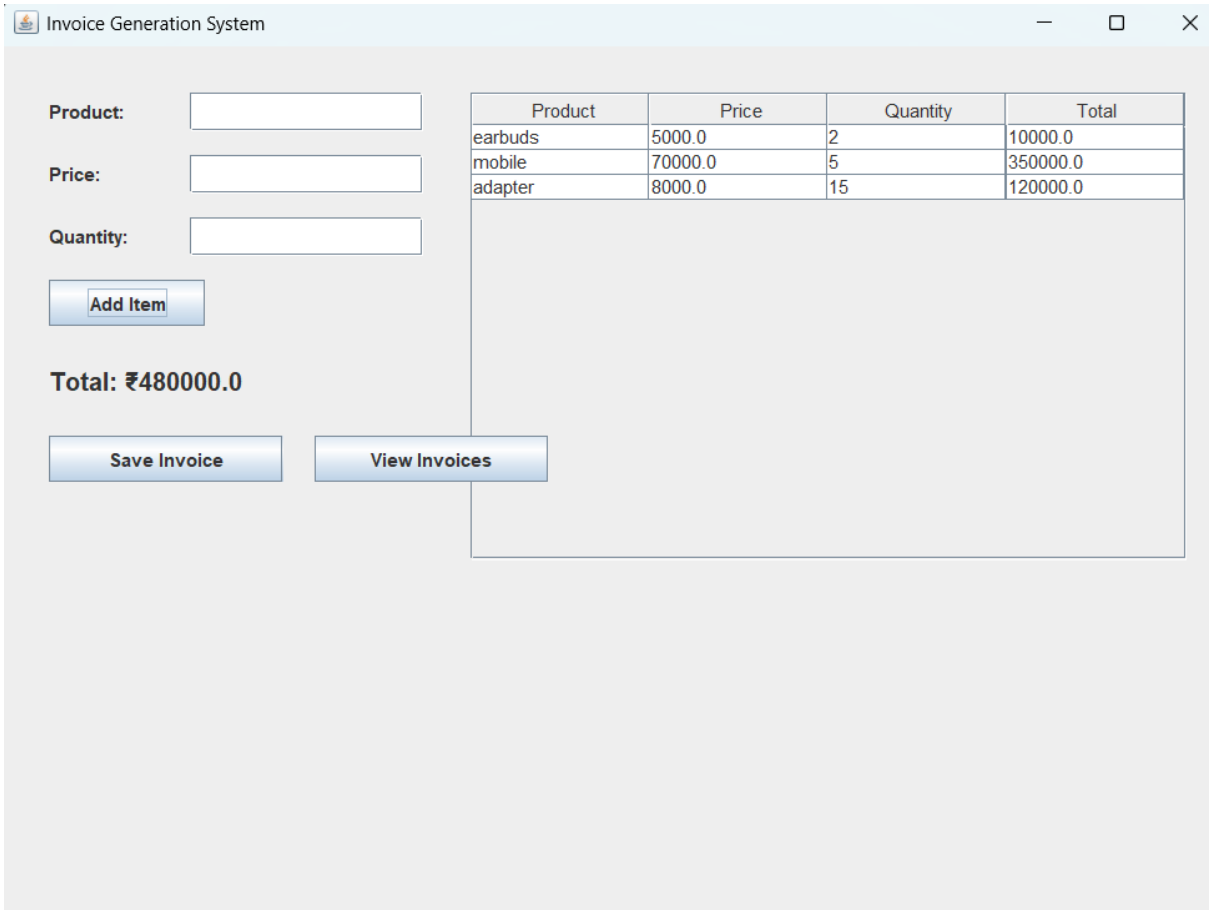
```
        DefaultTableModel viewModel = new DefaultTableModel(new String[]{"ID", "Product",
"Price", "Qty", "Total"}, 0);
        JTable viewTable = new JTable(viewModel);
        JScrollPane scroll = new JScrollPane(viewTable);
        viewFrame.add(scroll);

        try {
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM invoices");

            while (rs.next()) {
                viewModel.addRow(new Object[]{
                    rs.getInt("id"),
                    rs.getString("product_name"),
                    rs.getDouble("price"),
                    rs.getInt("quantity"),
                    rs.getDouble("total")
                });
            }

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Error fetching invoices: " + e.getMessage());
        }

        viewFrame.setVisible(true);
    }
```

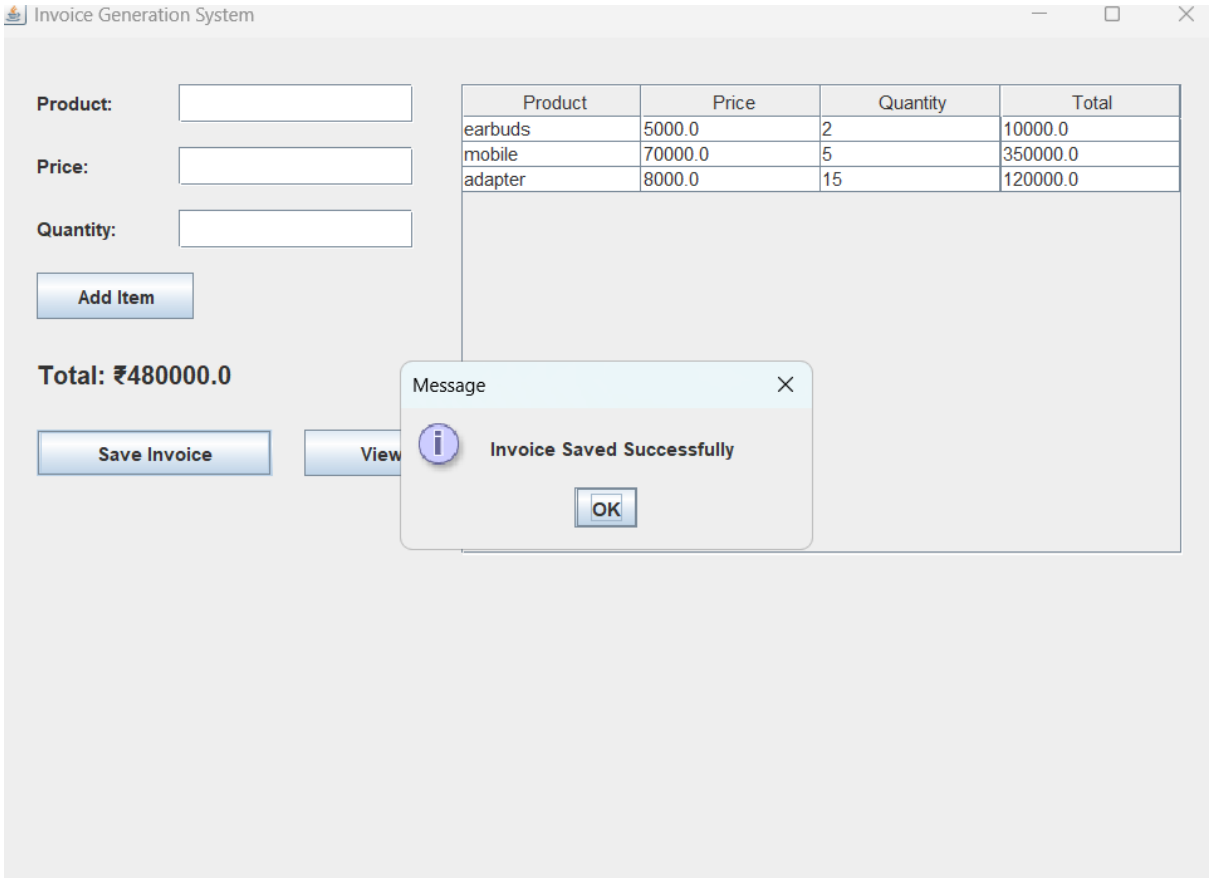
```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new InvoiceSystem().setVisible(true));
    }
}
```

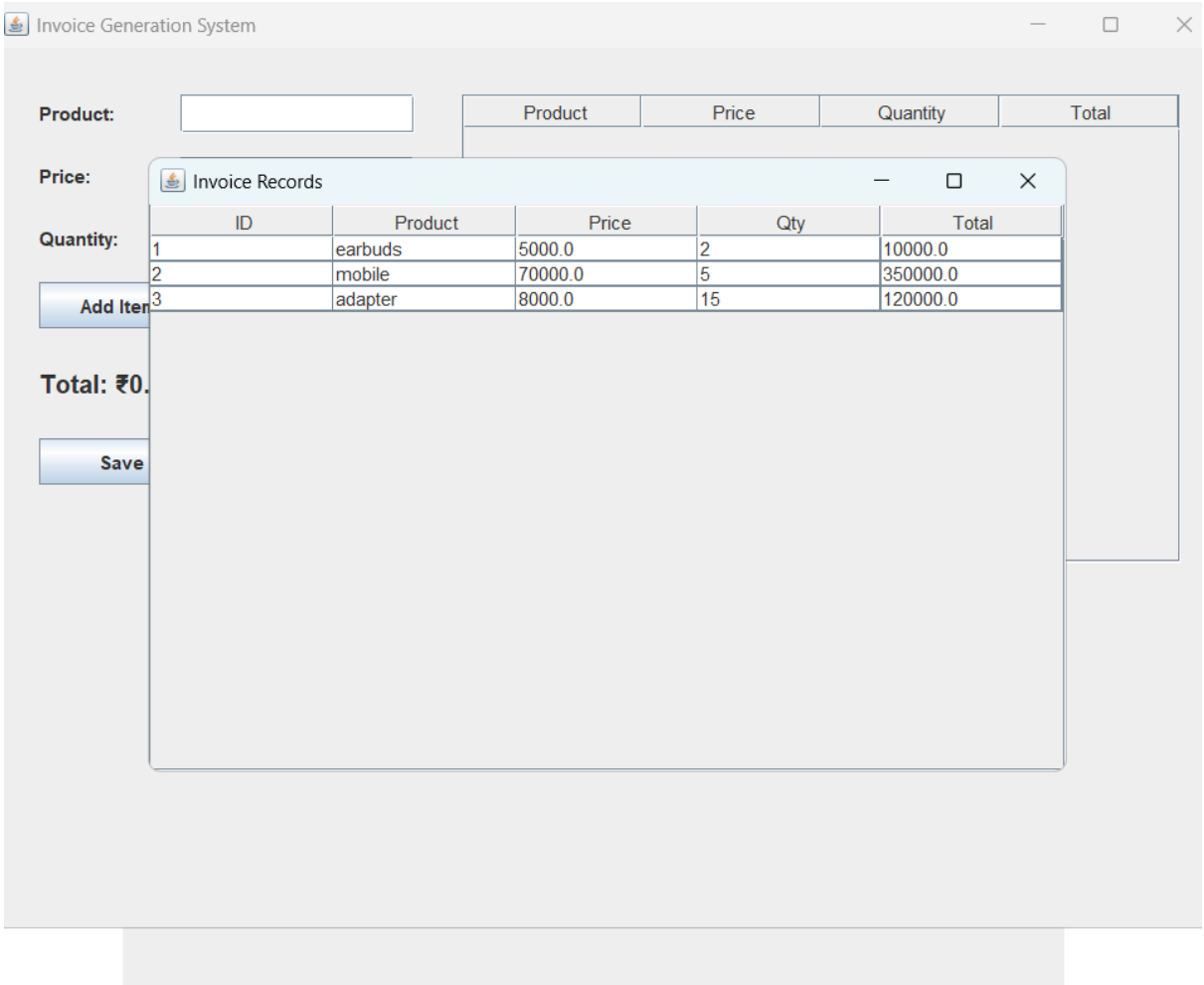
DBConnection.java

```
package assign5pkg;



import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    public static Connection getConnection() {
        try {
            String url = "jdbc:mysql://localhost:3306/user1";
            String user = "root";
            String pass = "admin@123";
            return DriverManager.getConnection(url, user, pass);
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Output:


Invoice Generation System
Product:
Price:
Quantity:
Add Item
Total: ₹480000.0
Save Invoice
View Invoices
Product
Price
Quantity
Total
earbuds
5000.0
2
10000.0
mobile
70000.0
5
350000.0
adapter
8000.0
15
120000.0


Invoice Generation System
Product:
Price:
Quantity:
Add Item
Total: ₹480000.0
Save Invoice
View
Product
Price
Quantity
Total
earbuds
5000.0
2
10000.0
mobile
70000.0
5
350000.0
adapter
8000.0
15
120000.0
Message
Invoice Saved Successfully
OK


Invoice Generation System
Product:
Price:
Quantity:
Add Iten
Total: ₹0.
Save
Product
Price
Quantity
Total
Invoice Records
ID
Product
Price
Qty
Total
1
earbuds
5000.0
2
10000.0
2
mobile
70000.0
5
350000.0
3
adapter
8000.0
15
120000.0